MINI PROJECT - SYNOPSIS

Title:

STUDY REPOSITORY - A Web Platform for Sharing Academic Resources

Submitted by:

1. Krishna Bobade (1272240613) - Team Lead
2. Parth Deshmukh (1272240432)
3. Jasnoor Singh (1272240588)

Guided by:

Aparna Kulkarni Madam

Pradeep Tiwari Sir

MIT World Peace University - Pune

Department of Computer Applications

SYBCA-B

Academic Year: 2025–2026

Date: 1 February 2026

Index

1. Nomenclature / Notations

The following abbreviations are used throughout the synopsis:

Abbreviation	Full Form / Meaning
PBL	Project-Based Learning
UI	User Interface
UX	User Experience
DBMS	Database Management System
CRUD	Create, Read, Update, Delete operations
LMS	Learning Management System
SQL	Structured Query Language
OTP	One-Time Password (if used for verification)
REST API	Representational State Transfer Application Programming Interface
JWT	JSON Web Token (for authentication, if used)
SRS	Software Requirements Specification

2. Abstract

This synopsis presents the design and planned development of “Study Repository,” a web-based platform that enables students to upload, share, search, and access academic resources such as lecture notes, assignments and solutions, lab manuals, previous year question papers, presentations, syllabus PDFs, study guides, e-books, and project reports. The proposed system promotes peer-to-peer learning, reduces study gaps, and improves academic support among students by centralizing resources and enabling quality signals through ratings and feedback. The platform emphasises usability, discoverability, and data integrity with key features including authenticated access restricted to institutional email IDs, metadata-rich resource uploads with tags and subjects, fast search and multi-criteria filtering, user ratings and comments, basic analytics of highly accessed materials, and security controls such as file-type and size validation, access control, and parameterised queries to prevent SQL injection. The initial prototype is planned with a static front-end hosted on GitHub Pages and local storage for demonstration, with future extension towards a full stack implementation using Node.js and a relational DBMS (MySQL/PostgreSQL). The end goal is a reliable, student-centric repository that saves time, encourages contribution, and builds a sustainable culture of collaborative learning.

3. Motivation

Students frequently spend significant time locating accurate and up-to-date study materials scattered across messaging groups, personal drives, and informal channels, leading to duplication, version confusion, and unequal access. Seniors often possess well-curated notes and solutions that juniors cannot find when needed, and college-level repositories, where present, are not always easy to search or contribute to. A dedicated, well-indexed Study Repository addresses these pain points by providing a single, trustworthy source of course-specific content with transparent provenance (uploader identity), quality signals (ratings, comments), and metadata (course, semester, subject, file type). By enabling secure uploads and controlled access for verified students, the system fosters peer support, reduces study gaps, and supports continuous academic improvement. Moreover, lightweight analytics can help instructors and coordinators identify high-demand topics and resource gaps, guiding targeted content creation and revision.

4. Literature Review (Comparative Table)

Table 1: Comparative review of prevalent approaches for sharing study materials (pros and cons).

Existing Approach / Platform	Pros	Cons
Messaging Groups (WhatsApp/Telegram)	Fast, familiar; quick broadcast; low entry barrier.	Poor search and organisation; content quickly buried; versioning issues; no quality signals; file size/type limits.
Shared Drives (Google Drive/OneDrive)	Centralised storage; link-based sharing; supports folders and permissions.	Discoverability depends on link circulation; inconsistent metadata; prone to clutter; limited peer feedback.
College LMS (Moodle/Google Classroom)	Course-level access control; supports assignments and grading; institutional backing.	Instructor-centric; limited student-to-student sharing; variable UX; search across courses may be weak.
GitHub Repositories/Pages	Version control; transparency; static site hosting for front-end prototypes.	Not optimised for binary academic files; contributor learning curve; requires separate back-end for dynamic features.
Email-based Sharing	Direct, simple; works offline for recipients once downloaded.	Inefficient; duplicates; no common catalog; difficult to maintain or update; lacks discoverability.

5. Problem Formulation and Objectives

Problem Statement: Students need a secure, searchable, and collaborative platform to upload, discover, and evaluate study resources across courses and semesters. Existing solutions either lack robust search and metadata, do not support peer contribution effectively, or present security and data-quality limitations.

General Objective: To develop a web-based Study Repository that centralises academic resources and enables students to upload, share, access, and evaluate materials efficiently and securely.

Specific Objectives:

• Implement authenticated access restricted to institutional email IDs.
• Provide resource uploading with metadata (title, subject, course, semester, tags, file type).
• Enable efficient search and filtering by subject, file type, course, semester, and uploader.
• Allow users to rate and comment on resources to surface quality content.
• Enforce security controls (file validation, access control, parameterised queries) to protect data.
• Provide basic analytics (e.g., most downloaded resources, most active subjects) for continuous improvement.
• Design an intuitive user interface covering login, home, upload, browse, my files, and profile pages.

6. Methodology / Planning of Work

1. Requirement Analysis

Gather functional and non-functional requirements from students and coordinators; define user roles, use cases (upload, browse, search, rate, comment), and acceptance criteria; draft an SRS.

2. System Design

Prepare architecture diagrams; design ER model and relational schema; define API endpoints; plan access control; sketch UI/UX using wireframes (Figma as an alternative).

3. Technology Stack Selection

Front-end: HTML, CSS, JavaScript with Bootstrap/Tailwind. Back-end: Node.js, ExpressJS, Database: MySQL/PostgreSQL; Prototype hosting: GitHub Pages (front-end), with local storage during demonstration, Django for backend, Python, Vercel for hosting, ReactJS.

4. Data Model & Validation

Define tables for users, resources, tags, ratings, comments, and analytics; enforce constraints; implement server-side validation for file type and size; parameterised queries to prevent SQL injection.

5. Authentication & Authorisation

Student sign-up/login restricted to college email; password hashing; session or token-based auth; role-based permissions for upload/download.

6. Core Feature Implementation

Resource upload with metadata; browse catalogue; full-text search and filters; download tracking; ratings and comment threads with moderation capabilities.

7. Analytics Module

Aggregate downloads, views, ratings by subject/file type; dashboards to surface most-accessed resources and active users/subjects while respecting privacy.

8. Security Hardening

Input sanitisation; parameterised SQL; rate limiting for uploads; access control checks; secure file handling; basic audit logs.

9. UI Implementation

Responsive pages: Login, Home, Upload, Browse, My Files, My Profile; consistent navigation; form hints and validation feedback.

10. Testing & Quality Assurance

Unit tests for server logic; integration tests for critical flows (upload, search, download); usability testing with student volunteers; performance checks for search/filter operations.

11. Deployment & Demonstration

Host the front-end on GitHub Pages; demonstrate functionality with local back-end and storage; prepare user guide and screencast for evaluation.

12. Documentation

Create technical documentation (README, API specification, DB schema), user manual, and final report.

7. Facilities Required for Proposed Work

Software:

• Operating System: Windows/Linux/macOS

• Front-end: HTML, CSS, JavaScript, Bootstrap, Tailwind CSS

• Back-end: Node.js (with Express.js)

• Database: MySQL or PostgreSQL

• Tools: Git and GitHub (GitHub Pages for front-end hosting), Figma (for UI/UX wireframing, optional), VS Code, Postman for API testing

Hardware:

• Development machine with multi-core CPU, ≥8 GB RAM, and stable internet connectivity

• Sufficient local storage for resource files during prototype demonstrations

8. Bibliography / References (IEEE Style)

[1] Jasnoor Singh, Parth Deshmukh and Krishna Bobade, “Study Repository - PBL Presentation,” 2025.

[2] Node.js Foundation, “Node.js Documentation,” 2025.

[3] MySQL, “MySQL 8.0 Reference Manual,” 2025.

[4] PostgreSQL Global Development Group, “PostgreSQL 16 Documentation,” 2025.

[5] Bootstrap, “Bootstrap Documentation,” 2025.

[6] Tailwind Labs, “Tailwind CSS Documentation,” 2025.

[7] GitHub, “GitHub Pages Documentation,” 2025.

[8] OWASP, “Injection Prevention Cheat Sheet,” 2025.